

INSTITUTE OF ENGINEERING AND MANAGEMENT, KOLKATA

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

TIER 2: ADVANCED USER INTERACTIONS

Assignment 4: JavaScript Alerts and Confirms

Name	Ashmita Saha
Enrollment No.	12023002028118
Department	CSE (AI & ML)
Course	Python for Automation
Academic Session	2023-2027
Submission Date	11 Sept, 2026

Lab Experiment Report

Module: Tier 2 - Advanced User Interactions | Experiment: JavaScript Alerts and Confirms

1. MODULE NAME & EXPERIMENT TITLE

Module Name: Tier 2: Advanced User Interactions (Focus: Handling complex web behaviors like overlays, pop-ups, and data structures)

Experiment Title: Assignment 4: JavaScript Alerts and Confirms

2. PROBLEM STATEMENT

Automating web interactions involving JavaScript pop-up modal dialogs (Alerts, Confirms, and Prompts) requires explicit control as these pop-ups are native browser UI elements outside the standard DOM tree. The task is to automate the handling of three distinct types of JavaScript alerts on a practice web application (https://the-internet.herokuapp.com/javascript_alerts):

- Trigger a standard **JS Alert**, read its text, and accept it.
- Trigger a **JS Confirm** dialog, read its text, and dismiss/cancel it.
- Trigger a **JS Prompt** dialog, input text into the prompt box using `driver.switch_to.alert.send_keys()`, and submit/accept it.

3. OBJECTIVE

- To master handling browser-level modal alerts, confirmation boxes, and prompt boxes using Selenium WebDriver with Python.
- To utilize `WebDriverWait` along with `expected_conditions` to wait dynamically until pop-up alerts appear before switching control to them.
- To execute alert operations using `driver.switch_to.alert`, specifically invoking `.text`, `.accept()`, `.dismiss()`, and `.send_keys()` methods seamlessly.

4. TOOLS, SOFTWARE, AND CONCEPTS USED

- **Programming Language:** Python 3.x
- **Automation Library:** Selenium WebDriver (v4.x)
- **Browser Engine:** Google Chrome & ChromeDriver
- **Virtual Environment:** Python Virtual Environment (`.venv`)
- **Key Concepts:**
 - **Explicit Waits (`WebDriverWait`):** Synchronizing script execution dynamically with expected conditions such as `element_to_be_clickable` and `alert_is_present`.
 - **Alert Switching (`driver.switch_to.alert`):** Transferring driver focus from the HTML DOM to the active modal alert box.

◦ Alert Interface Methods:

- `alert.text`: Extracting visible text from the pop-up box.
- `alert.accept()`: Simulating a click on the "OK" button.
- `alert.dismiss()`: Simulating a click on the "Cancel" button.
- `alert.send_keys("text")`: Injecting keystrokes into prompt input fields.

5. SOURCE CODE / IMPLEMENTATION STEPS

The code below demonstrates full automation for handling JS alerts, confirmation dialogs, and text prompt input:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
import time

driver = webdriver.Chrome()
wait = WebDriverWait(driver, 10)

driver.get("https://the-internet.herokuapp.com/javascript_alerts")
driver.maximize_window()

wait.until(EC.element_to_be_clickable((By.XPATH, "//button[text()='Click for JS Alert']"))).click()
alert = wait.until(EC.alert_is_present())
print("Alert message:", alert.text)
alert.accept()
print("JavaScript Alert accepted successfully.")

wait.until(EC.element_to_be_clickable((By.XPATH, "//button[text()='Click for JS Confirm']"))).click()
confirm = wait.until(EC.alert_is_present())
print("Confirm message:", confirm.text)
confirm.dismiss()
print("Confirm box dismissed successfully.")

wait.until(EC.element_to_be_clickable((By.XPATH, "//button[text()='Click for JS Prompt']"))).click()
wait.until(EC.alert_is_present())
driver.switch_to.alert.send_keys("Selenium Lab Report")
print("Text entered in Prompt box: Selenium Lab Report")
driver.switch_to.alert.accept()
print("Prompt box submitted successfully.")

time.sleep(2)
driver.quit()
print("Assignment 4 completed successfully.")
```

Figure 1: Source Code Implementation (Asst_4.py in VS Code Editor)

6. OUTPUT WITH CLEARLY LABELLED SCREENSHOTS

The PowerShell terminal log confirms successful activation of the virtual environment, sequential execution of each alert interaction step, print outputs for alert texts, and clean process termination.

```

• PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned)
; (& "c:\Users\heyas\OneDrive\Desktop\Wipro Selenium\.venv\Scripts\Activate.ps1")
• (.venv) PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium> python Asst_4.py
Alert message: I am a JS Alert
JavaScript Alert accepted successfully.
Confirm message: I am a JS Confirm
Confirm box dismissed successfully.
Text entered in Prompt box: Selenium Lab Report
Prompt box submitted successfully.
Assignment 4 completed successfully.
❖ (.venv) PS C:\Users\heyas\OneDrive\Desktop\Wipro Selenium>

```

Figure 2: Execution Output in PowerShell Terminal

7. BRIEF RESULT, OBSERVATION, AND CONCLUSION

Result & Observations

- **JS Alert:** Clicked "Click for JS Alert", captured message *"I am a JS Alert"*, and accepted it via `alert.accept()`.
- **JS Confirm:** Clicked "Click for JS Confirm", captured message *"I am a JS Confirm"*, and dismissed it via `confirm.dismiss()`.
- **JS Prompt:** Clicked "Click for JS Prompt", transmitted text string "Selenium Lab Report" using `driver.switch_to.alert.send_keys()`, and accepted the dialog.
- **Synchronization:** Utilizing explicit waits with `EC.alert_is_present()` completely prevented premature context switching or `NoAlertPresentException` errors.

Conclusion:

The lab experiment for **Tier 2 Assignment 4: JavaScript Alerts and Confirms** was completed successfully. The python automation script interacted with native browser pop-up windows cleanly and reliably, satisfying all lab requirements.